

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

二叉树基础与算法应用

Binary Tree Fundamentals and Algorithms

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia
2026 年 11 月 2 日

目录

1. 二叉树基础概念
2. 二叉树的遍历
3. 层序遍历与树的应用
4. 二叉搜索树
5. 总结



目录

1 二叉树基础概念

2 二叉树的遍历

3 层序遍历与树的应用

4 二叉搜索树

5 总结

什么是二叉树

二叉树：每个节点最多有两个子节点的树结构。

核心术语

- **根节点**：最顶端的节点，没有父节点。
- **叶子节点**：没有子节点的节点。
- **左子树 / 右子树**：每个节点的两个子节点，左右不可互换。

两种特殊的二叉树

- **满二叉树**：除了叶子节点，每个节点都有两个子节点。
- **完全二叉树**：除最后一层外全满，且最后一层靠左对齐。

完全二叉树是最常用的结构，可以直接用数组存

二叉树的存储方式

方式一：链式存储（竞赛常用）

- 用结构体表示节点，包含左右子节点的下标或指针。
- 适合任意形态的二叉树。

方式二：数组存储（完全二叉树）

- 根节点存在下标 1。
- 节点 i 的左孩子是 $2i$，右孩子是 $2i+1$。
- 优点：无需存左右指针，父子关系直接算。

竞赛中多用静态数组：`l[N]` 和 `r[N]` 存左右儿子

代码：静态数组建树


```cpp
const int N = 1e5 + 10;
int l[N], r[N], val[N], root;

// 建树：输入每个节点的左右儿子编号，0 表示空
void build(int n) {
    for (int i = 1; i <= n; i++) {
        cin >> val[i] >> l[i] >> r[i];
        // 如果要找根，可以用一个标记数组
    }
}
```

`l[i]` 和 `r[i]` 存的是子节点的下标，0 表示空节点

目录

1 二叉树基础概念

2 二叉树的遍历

3 层序遍历与树的应用

4 二叉搜索树

5 总结



什么是遍历

遍历：按某种规则，不重不漏地访问树中的每个节点。

最常见的三种 DFS 遍历

- **前序遍历**：根、左、右。
- **中序遍历**：左、根、右。
- **后序遍历**：左、右、根。

一种 BFS 遍历

- **层序遍历**：从根开始，一层一层从上往下，从左往右访问。

三种 DFS 遍历的代码结构完全一样，只是访问顺序不同

例题：前序遍历

题目：给定一棵二叉树，输出它的前序遍历序列。

示例树

- 根为 1，左孩子 2，右孩子 3
- 节点 2 的左孩子 4，右孩子 5
- 节点 3 无子节点
- 输出：$1, 2, 4, 5, 3$

前序遍历：解法

核心思路：按照“根 $\rightarrow$ 左 $\rightarrow$ 右”的顺序，用递归的方式访问每个节点。

关键步骤

- 第一步：传入当前节点 u。
- 第二步：如果 u 为空，直接返回。
- 第三步：访问根节点 u（输出）。
- 第四步：递归左子树 `dfs(l[u])`。
- 第五步：递归右子树 `dfs(r[u])`。

先输出，再往左走，再往右走

前序遍历：代码


```
1 void preOrder(int u) {
2     if (u == 0) return;          // 遇到空节点，结束递归
3     cout << u << " ";            // 根：先访问
4     preOrder(l[u]);              // 左：递归左子树
5     preOrder(r[u]);              // 右：递归右子树
6 }
```

改变 cout 的位置，就能变成另外两种遍历

例题：中序遍历

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

题目：给定一棵二叉树，输出它的中序遍历序列。

示例树

- 根为 1，左孩子 2，右孩子 3
- 节点 2 的左孩子 4，右孩子 5
- 节点 3 无子节点
- 输出：$4, 2, 5, 1, 3$

中序遍历：解法

核心思路：按照“左 $\rightarrow$ 根 $\rightarrow$ 右”的顺序访问。

关键步骤

- 第一步：如果 u 为空，直接返回。
- 第二步：先递归左子树 `dfs(l[u])`。
- 第三步：访问根节点 u（输出）。
- 第四步：递归右子树 `dfs(r[u])`。

先一直往左走到底，回来时再输出，最后走右边

中序遍历：代码


```
1 void inOrder(int u) {
2     if (u == 0) return;
3     inOrder(l[u]);              // 左：递归左子树
4     cout << u << " ";           // 根：回来时再输出
5     inOrder(r[u]);              // 右：递归右子树
6 }
```

二叉搜索树的中序遍历一定是一个严格递增的序列

例题：后序遍历

题目：给定一棵二叉树，输出它的后序遍历序列。

示例树

- 根为 1，左孩子 2，右孩子 3
- 节点 2 的左孩子 4，右孩子 5
- 节点 3 无子节点
- 输出：$4, 5, 2, 3, 1$

后序遍历：解法

核心思路：按照“左 $\rightarrow$ 右 $\rightarrow$ 根”的顺序访问。

关键步骤

- 第一步：如果 u 为空，直接返回。
- 第二步：先递归左子树 `dfs(l[u])`。
- 第三步：再递归右子树 `dfs(r[u])`。
- 第四步：最后访问根节点 u（输出）。

左右孩子全处理完了，最后再输出根节点

后序遍历：代码


```cpp
void postOrder(int u) {
    if (u == 0) return;
    postOrder(l[u]);          // 左：递归左子树
    postOrder(r[u]);          // 右：递归右子树
    cout << u << " ";         // 根：最后访问
}
```

后序遍历常用于先算子树答案，再合并到当前节点（如树形 DP）

目录

1 二叉树基础概念

2 二叉树的遍历

3 层序遍历与树的应用

4 二叉搜索树

5 总结

例题：层序遍历

题目：给定一棵二叉树，按层从上到下、从左到右输出节点。

示例树

- 根为 1，左孩子 2，右孩子 3
- 节点 2 的左孩子 4，右孩子 5
- 节点 3 无子节点
- 输出：$1, 2, 3, 4, 5$

层序遍历：解法

核心思路：使用队列，每次取出队首元素，并将其左右儿子入队。

关键步骤

- 第一步：把根节点入队。
- 第二步：当队列不为空时，取队首 u，输出。
- 第三步：如果左儿子不为空，入队。
- 第四步：如果右儿子不为空，入队。

层序遍历就是按层扫描的 BFS，保证按距离远近访问

层序遍历：代码


```cpp
void levelOrder(int root) {
    queue<int> q;
    if (root != 0) q.push(root);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        cout << u << " ";
        if (l[u] != 0) q.push(l[u]);   // 左儿子入队
        if (r[u] != 0) q.push(r[u]);   // 右儿子入队
    }
}
```

例题：二叉树的深度

题目：给定一棵二叉树，求它的最大深度。深度从根节点开始算，根节点深度为 1。

示例

- 根为 1，左孩子 2，右孩子 3
- 节点 2 的左孩子 4，右孩子 5
- 节点 3 无子节点
- 最大深度为 3（路径 $1 \to 2 \to 4$）

二叉树的深度：解法

核心思路：当前节点的深度 $=$ 左右子树深度的最大值 $+\ 1$。

关键步骤

- 第一步：如果 u 为空，返回 0。
- 第二步：递归求左子树深度 `lh = dfs(l[u])`。
- 第三步：递归求右子树深度 `rh = dfs(r[u])`。
- 第四步：返回 `max(lh, rh) + 1`。

树形 DP 的雏形：先求子树答案，再合并到当前节点

二叉树的深度：代码

```
int getDepth(int u) {
    if (u == 0) return 0;           // 空节点深度为 0
    int lh = getDepth(l[u]);        // 左子树深度
    int rh = getDepth(r[u]);        // 右子树深度
    return max(lh, rh) + 1;         // 当前节点深度 = max + 1
}
```

目录

1 二叉树基础概念

2 二叉树的遍历

3 层序遍历与树的应用

4 二叉搜索树

5 总结



什么是二叉搜索树

二叉搜索树（BST）：一种特殊的二叉树，满足以下性质：

- 左子树所有节点的值，都小于根节点的值。
- 右子树所有节点的值，都大于根节点的值。
- 左右子树也分别是二叉搜索树。

最核心的性质

- 对 BST 进行**中序遍历**，结果一定是一个**严格递增**的序列。
- 插入、查找、删除的平均复杂度为 $O(\log n)$。

左小右大，中序有序

例题：验证二叉搜索树

题目：给定一棵二叉树的根节点，判断它是否是二叉搜索树。

示例

- 根为 2，左孩子 1，右孩子 3
- 满足 BST 性质，输出 true。

反例

- 根为 5，左孩子 1，右孩子 4
- 节点 4 的左孩子 3，右孩子 6
- 不满足 BST 性质，输出 false。

验证二叉搜索树：解法

核心思路：BST 的中序遍历是严格递增的，可以在中序遍历时记录前一个节点的值，判断是否递增。

关键步骤

- 第一步：定义一个全局变量 `prev` 记录上一个节点值，初始极小。
- 第二步：中序遍历，先访问左子树。
- 第三步：判断当前节点值是否大于 `prev`，不大于则返回 false。
- 第四步：更新 `prev`，再访问右子树。

利用中序遍历的有序性，是验证 BST 最常用的方法

验证二叉搜索树：代码


```cpp
long long prev = -1e18;            // 记录前一个节点的值

bool isValidBST(int u) {
    if (u == 0) return true;

    if (!isValidBST(l[u])) return false;  // 左

    if (val[u] <= prev) return false;     // 中：必须严格递增
    prev = val[u];

    if (!isValidBST(r[u])) return false;  // 右

    return true;
}
```

目录

1 二叉树基础概念

2 二叉树的遍历

3 层序遍历与树的应用

4 二叉搜索树

5 总结



总结：遍历方式对比

遍历方式	访问顺序	典型应用
前序遍历	根左右	复制树、找路径
中序遍历	左根右	BST 验证、求第 k 小
后序遍历	左右根	树形 DP、求深度、表达式求值
层序遍历	从前往后	求树的宽度、迷宫最短步

DFS 通用模板

- 递归边界：空节点返回。
- 递归逻辑：根据题目需求调整左、根、右的相对顺序。

总结：常见易错点

- 在静态数组存树时，忘记判断 `l[u]` $=0$! 和 `r[u]` $=0$! 就直接递归。
- 递归函数缺少 `if (u == 0) return;` 导致死循环或越界。
- 在 BST 中，左子树的最大值可能大于根节点的值，单看左右儿子是不够的，要用中序或上下界约束。
- 数组建树时，完全二叉树的编号是从 1 开始的，如果是 0 开始，左右孩子就是 $2i+1$ 和 $2i+2$。

遇到二叉树，先想空节点，再想遍历顺序

附录：完整测试代码


```cpp
#include <iostream>
using namespace std;

const int N = 1e5 + 10;
int l[N], r[N], val[N];

void preOrder(int u) {
    if (!u) return;
    cout << u << " ";
    preOrder(l[u]);
    preOrder(r[u]);
}

int main() {
    int n; cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> val[i] >> l[i] >> r[i];
    }
```

附录：完整测试代码


```
19      preOrder(1);    // 假设 1 是根
20      return 0;
21  }
```